PORTADA

INSTITUTO TECNOLOGICO SUPERIOR DE RIOVERDE

Materia: Programación orientada a objetos

Practica: Sobrecarga de métodos y operadores en C++

Alumno: Luis Alejandro Domínguez Ramírez

Profesor: ING. José de Jesús Collazo Reyes

Fecha de entrega: 17/04/2026

¿Qué versiones del método mover existen?

Existen 3:

- mover()

-mover (char dirección)

-mover (int pasos)

¿Cuál se ejecuta en cada caso?

-p.mover (); → ejecuta mover()

-p.mover ('d'); → ejecuta mover(char)

-p.mover (3); → ejecuta mover(int)

¿Cómo decide el programa qué función usar?

-El número de parámetros

-El tipo de datos (char o int)

¿Qué hace el operador +?

-Crea un nuevo objeto

-Le suma puntos

-No modifica directamente el original (solo cuando lo reasignas)

¿Qué hace el operador +=?

-Modifica el objeto directamente

-Aumenta los puntos del mismo objeto

¿Qué cambia en el objeto?

-Posición (x, y) → cambia con mover

-Puntos → cambian con + y +=

INTRODUCCION

En esta práctica se desarrolló un programa en C++ que simula el comportamiento de un personaje tipo Pacman.
El objetivo fue comprender la sobrecarga de métodos y operadores, permitiendo que una misma función tenga diferentes comportamientos según los parámetros recibidos.

DESARROLLO

-Pacman.h

Pacman.h > ...

```
#ifndef PACMAN_H
#define PACMAN_H

#include <iostream>
using namespace std;

class Pacman {
private:
    int x, y;
    int puntos;

public:
    Pacman(int x, int y);

    void mostrar();


    void mover();
    void mover(char direccion);
    void mover(int pasos);


    Pacman operator+(int p);
    void operator+=(int p);
};

#endif
```

¿Qué se está mostrando?

Se muestra el archivo de encabezado Pacman.h, donde se declara la clase Pacman. En este archivo se pueden observar los atributos del objeto, así como los métodos y operadores que podrá utilizar.

¿Qué ocurre en ese momento?

En este punto del programa no se está ejecutando ninguna acción, ya que únicamente se están definiendo las características del objeto. Es decir, se está estableciendo qué datos tendrá el objeto y qué funciones podrá realizar, como moverse o acumular puntos.

¿Qué relación tiene con el programa?

Este archivo es la base del programa, ya que define la estructura del objeto principal que se utilizará en el resto del código.
Gracias a esta declaración, los otros archivos (como él .cpp y el main) pueden implementar y utilizar los métodos, incluyendo la sobrecarga de métodos y operadores que permiten diferentes comportamientos del objeto.

-Pacman.cpp

Pacman.cpp > ...

```cpp
#include "Pacman.h"

Pacman::Pacman(int x, int y) {
    this->x = x;
    this->y = y;
    puntos = 0;
    cout << "Pacman creado en posicion (" << x << "," << y << ")\n";
}

void Pacman::mostrar() {
    cout << "Posicion: (" << x << "," << y << ") | Puntos: " << puntos << endl;
}


void Pacman::mover() {
    x++;
    cout << "Movimiento por defecto\n";
}


void Pacman::mover(char direccion) {
    if (direccion == 'w') y--;
    else if (direccion == 's') y++;
    else if (direccion == 'a') x--;
    else if (direccion == 'd') x++;

    cout << "Movimiento con direccion\n";
}


void Pacman::mover(int pasos) {
    x += pasos;
    cout << "Movimiento multiple\n";
}


Pacman Pacman::operator+(int p) {
```

¿Qué se está mostrando?

Se muestra el archivo Pacman.cpp, el cual contiene la implementación de los métodos previamente declarados en el archivo de encabezado. Aquí se puede observar cómo funcionan los movimientos del personaje, así como la lógica de los operadores sobrecargados.

¿Qué ocurre en ese momento?

En este punto se define el comportamiento real del objeto. Es decir, se especifica qué sucede cuando se llama a cada método, por ejemplo:

-Cómo se mueve el personaje en cada versión de mover

-Cómo se suman los puntos con los operadores + y +=

Aquí es donde el programa pasa de solo tener una estructura a tener funcionalidad.

¿Qué relación tiene con el programa?

Este archivo es esencial porque permite que el objeto Pacman funcione correctamente. Sin esta implementación, los métodos no tendrían acciones definidas. Además, aquí se hace evidente la sobrecarga, ya que cada versión del método mover realiza una acción diferente dependiendo de los parámetros.

-Main.cpp

main.cpp > main()

```cpp
#include "Pacman.h"

int main() {

    Pacman p(0,0);

    p.mostrar();

    p.mover();
    p.mostrar();

    p.mover('d');
    p.mostrar();

    p.mover(3);
    p.mostrar();

    p = p + 10;
    p.mostrar();

    p += 5;
    p.mostrar();

    return 0;
}
```

¿Qué se está mostrando?

Se muestra el archivo main.cpp**,** que es el programa principal donde se crea el objeto de tipo Pacman y se utilizan sus métodos y operadores.

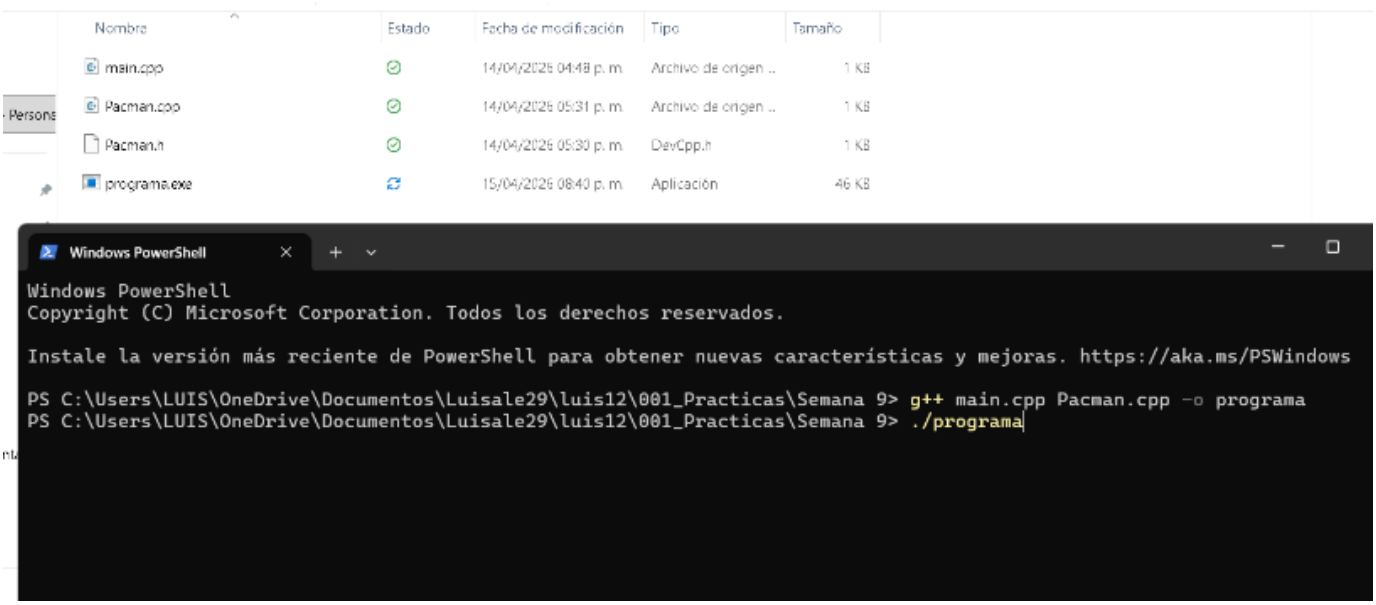
¿Qué ocurre en ese momento?

En este punto el programa se ejecuta. Se crea un objeto Pacman y se llaman los diferentes métodos sobrecargados, así como los operadores.
Esto permite observar cómo el objeto cambia su posición y sus puntos conforme se interactúa con él.

¿Qué relación tiene con el programa?

Este archivo es el encargado de poner en funcionamiento todo lo definido anteriormente. Aquí se demuestra el uso de la sobrecarga de métodos y operadores, ya que se invocan distintas versiones de mover y se utilizan los operadores para modificar el estado del objeto.

COPILACION



Explicación; Se muestra la ventana de PowerShell donde se compila y ejecuta el programa en C++. Primero se usa el comando g++ main.cpp Pacman.cpp -o programa para generar el archivo ejecutable, y luego. /programa para correrlo.

En este momento se transforma el código en un programa funcional. Esto es importante porque permite comprobar que todo el código funciona correctamente y ejecutar el objeto Pacman.

EJECUCION DEL PROGRAMA

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\LUIS\OneDrive\Documentos\Luisale29\luis12\001_Practicas\Semana 9> g++ main.cpp Pacman.cpp -o programa
PS C:\Users\LUIS\OneDrive\Documentos\Luisale29\luis12\001_Practicas\Semana 9> ./programa
Pacman creado en posicion (0,0)
Posicion: (0,0) | Puntos: 0
Movimiento por defecto
Posicion: (1,0) | Puntos: 0
Movimiento con direccion
Posicion: (2,0) | Puntos: 0
Movimiento multiple
Posicion: (5,0) | Puntos: 0
Posicion: (5,0) | Puntos: 10
Posicion: (5,0) | Puntos: 15
PS C:\Users\LUIS\OneDrive\Documentos\Luisale29\luis12\001_Practicas\Semana 9>
```

Se muestra la ejecución del programa en la consola, donde se puede observar el comportamiento del objeto Pacman. Al inicio, el personaje se crea en la posición (0,0) con 0 puntos.

Después, se ejecutan los diferentes métodos sobrecargados mover (), lo que provoca cambios en la posición del objeto según la forma en que se invoca (sin parámetros, con dirección y con pasos).

Finalmente, se utilizan los operadores + y +=, lo que incrementa los puntos del personaje, pasando de 0 a 10 y luego a 15. Esto demuestra cómo cambia el estado del objeto durante la ejecución del programa.

Análisis

En este programa se usa la sobrecarga de métodos y de operadores en la clase Pacman.

La sobrecarga de métodos significa que un mismo método puede hacer cosas diferentes según lo que reciba. En este caso, el método mover tiene tres versiones: una sin parámetros que mueve a la derecha por defecto, otra que recibe una dirección (w, a, s, d) para moverse, y otra que recibe un número para avanzar varios pasos. Dependiendo de cómo se llame, el programa usa la versión correcta. La sobrecarga de operadores permite usar operadores como + y += con objetos. El operador + crea un nuevo objeto sumando puntos, mientras que += suma los puntos directamente al objeto original.

El método que se ejecuta depende de cómo se escriba en el main. Por ejemplo, p.mover () usa el método sin parámetros, p.mover ('d') usa el de dirección y p.mover (3) usa el de pasos. El estado del objeto va cambiando durante el programa. Empieza en (0,0) con 0 puntos, luego se mueve cambiando su posición, y después aumenta sus puntos con los operadores.

El flujo del programa empieza en main, donde se crea el objeto y se llaman los métodos. Luego se ejecuta el código en Pacman.cpp, se modifican los datos y se muestran los resultados en pantalla.

Conclusiones

En esta práctica aprendí cómo funciona la sobrecarga de métodos y de operadores en C++. Entendí que se puede usar el mismo nombre de función para hacer diferentes acciones dependiendo de los parámetros, y que también se pueden adaptar operadores como + y += para trabajar con objetos.

Una de las dificultades que tuve fue entender cómo el programa sabe qué método ejecutar y cómo funcionan los operadores sobrecargados, especialmente el operador + porque crea un nuevo objeto. También me costó un poco seguir el flujo del programa entre los diferentes archivos.

Este conocimiento es útil en la vida real porque permite hacer programas más organizados y fáciles de usar. Por ejemplo, en videojuegos o aplicaciones, se pueden manejar objetos de forma más natural, como mover personajes o modificar sus datos de manera sencilla usando métodos y operadores.